

# Actividad 2

**Alumno**

Alan Lam Páez

**Matrícula**

T03063809

**Materia**

Aplicaciones Interactivas

**Maestro(a)**

Cristopher Gerardo Gaytán Díaz

**Correo**

[al03063809@tecmilenio.mx](mailto:al03063809@tecmilenio.mx)

**Fecha**

20/08/2026

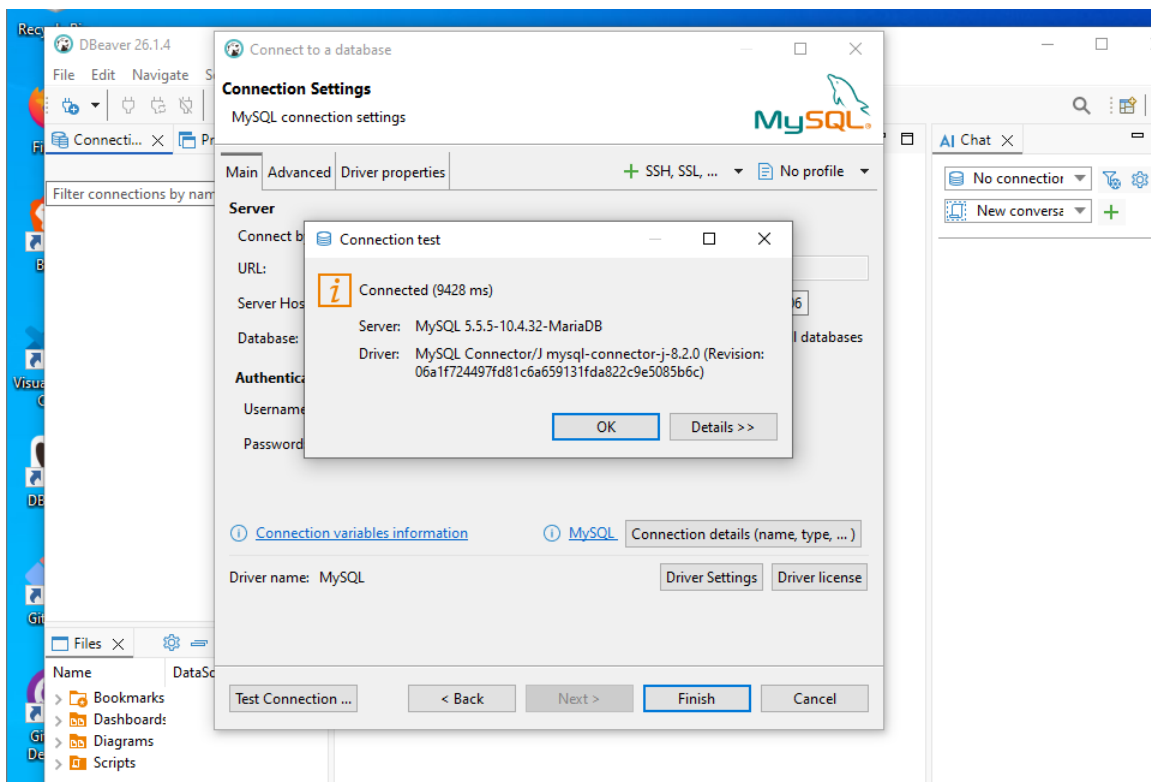
# Evidencias del proyecto

- Puede consultar los archivos del proyecto en el siguiente repositorio público de Github: <https://github.com/Alan-AFK1/API-act2>
- Si presenta problemas para visualizar los archivos, también es posible visualizar la carpeta del proyecto en **API-act2.zip**.
- Consulte la demostración del proyecto en video en la siguiente carpeta en la nube:

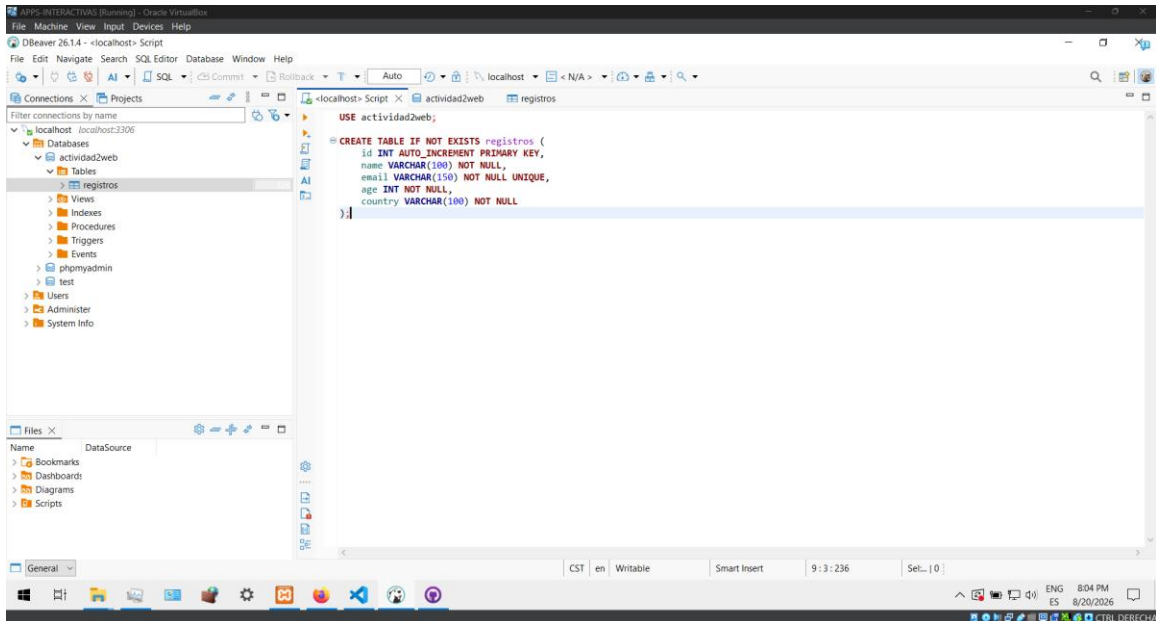
<https://drive.proton.me/urls/W5NW8NHQEW#WM5JgYMWg9o4>

## Proceso del proyecto

1. Asegurar la conexión de MariaDB (base de datos de XAMPP): Conectar una base de datos desde DBeaver > Se selecciono una base de datos tipo "MySQL" > Se dejaron los campos por defecto (puerto, usuario y contraseña) y se testeó la conexión:



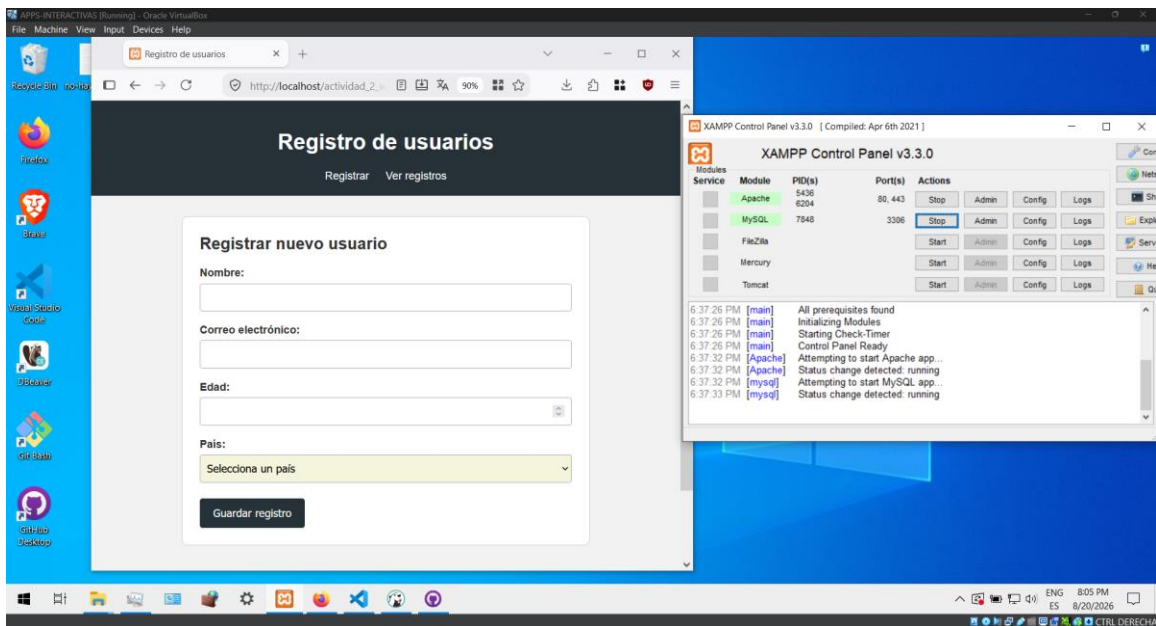
2. Se creo una base de datos manualmente desde DBeaver para *localhost* llamada "actividad2web", y se creo una tabla "registros" con el siguiente script:



**USE** actividad2web;

**CREATE TABLE IF NOT EXISTS** registros (  
    id **INT AUTO\_INCREMENT PRIMARY KEY**,  
    name **VARCHAR(100) NOT NULL**,  
    email **VARCHAR(150) NOT NULL UNIQUE**,  
    age **INT NOT NULL**,  
    country **VARCHAR(100) NOT NULL**  
);

3. La conexión al servidor local es exitosa:



4. Todas las conexiones a la BD se hacen con conexion.php:

```
<?php
```

```
$servidor = "localhost";
```

```
$usuario = "root";
```

```
$contrasena = "";
```

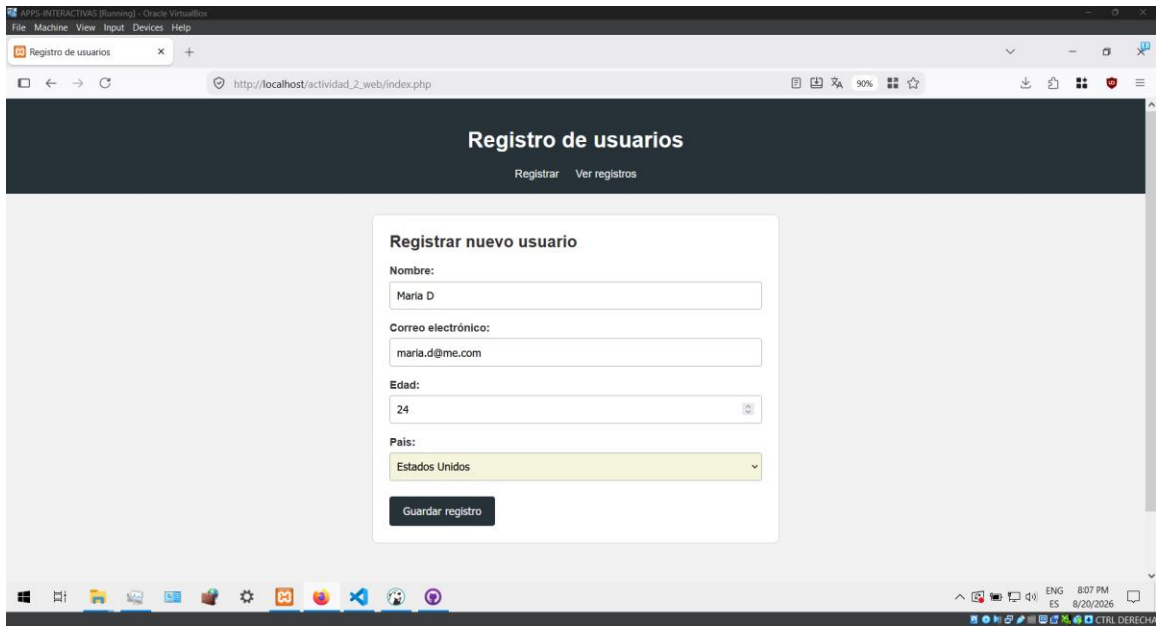
```
$base_de_datos = "actividad2web";
```

```
$conexion = new mysqli($servidor, $usuario, $contrasena,  
$base_de_datos);
```

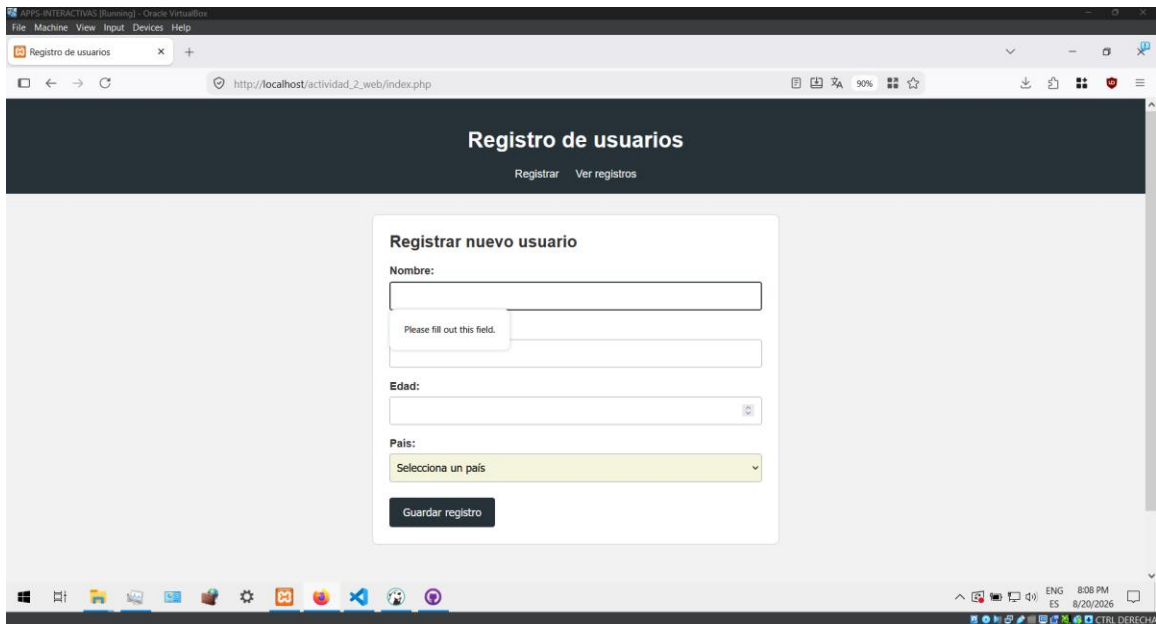
```
if ($conexion->connect_error) {  
    die("Error de conexión: " . $conexion->connect_error);  
}
```

```
$conexion->set_charset("utf8mb4");  
?>
```

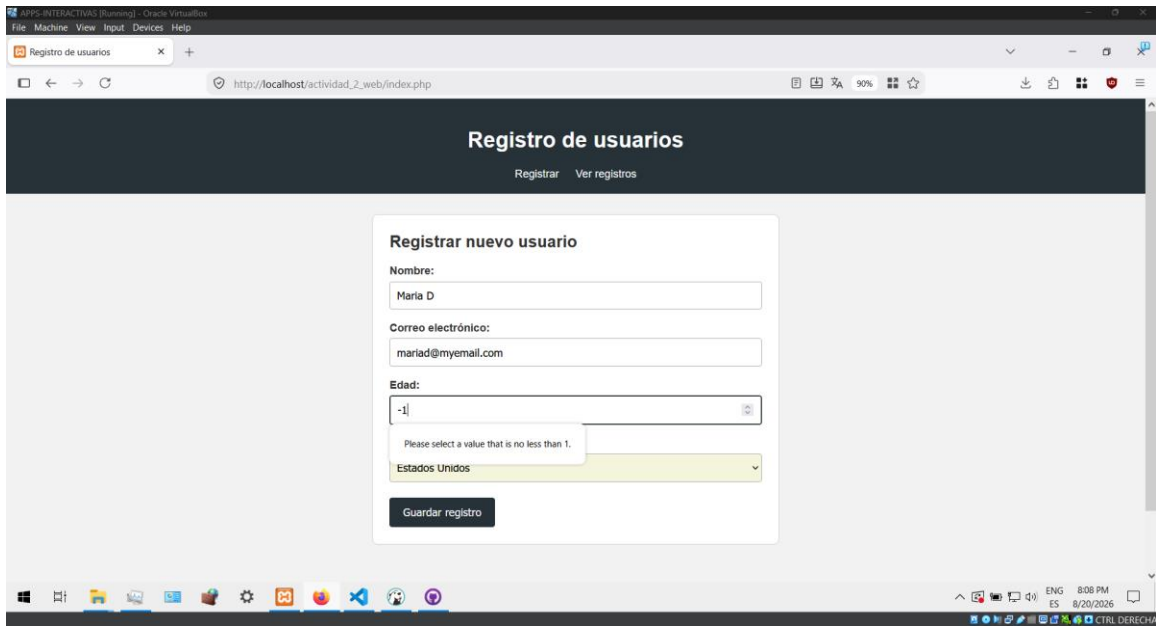
5. Ejemplo del uso del formulario:



a. Rellenar datos

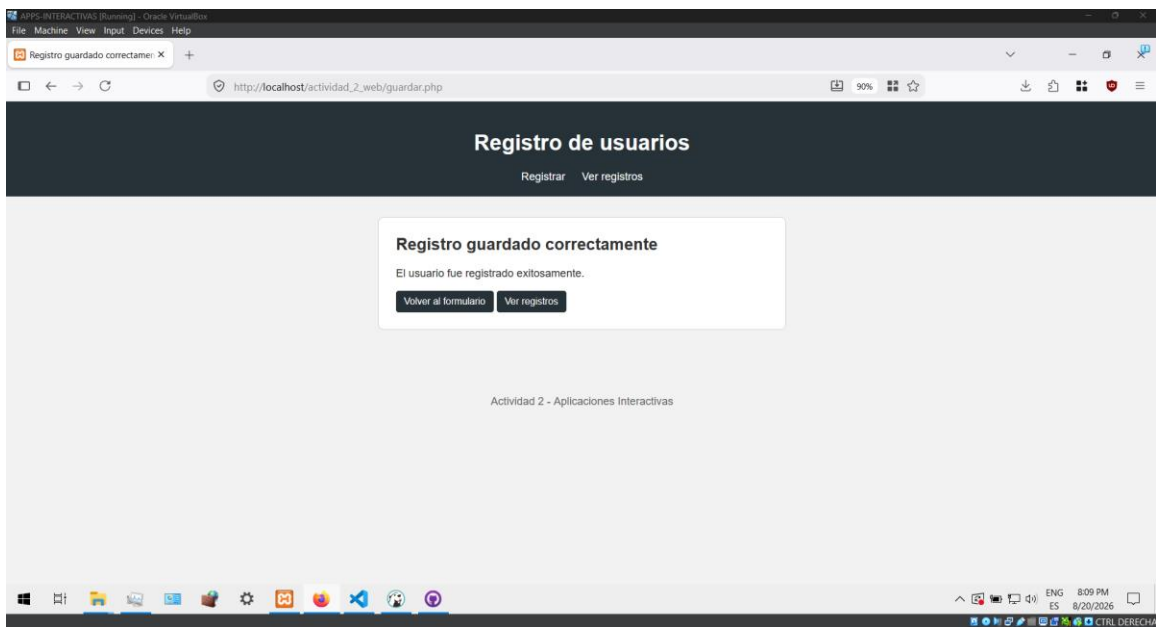


b. Todos los campos son necesarios

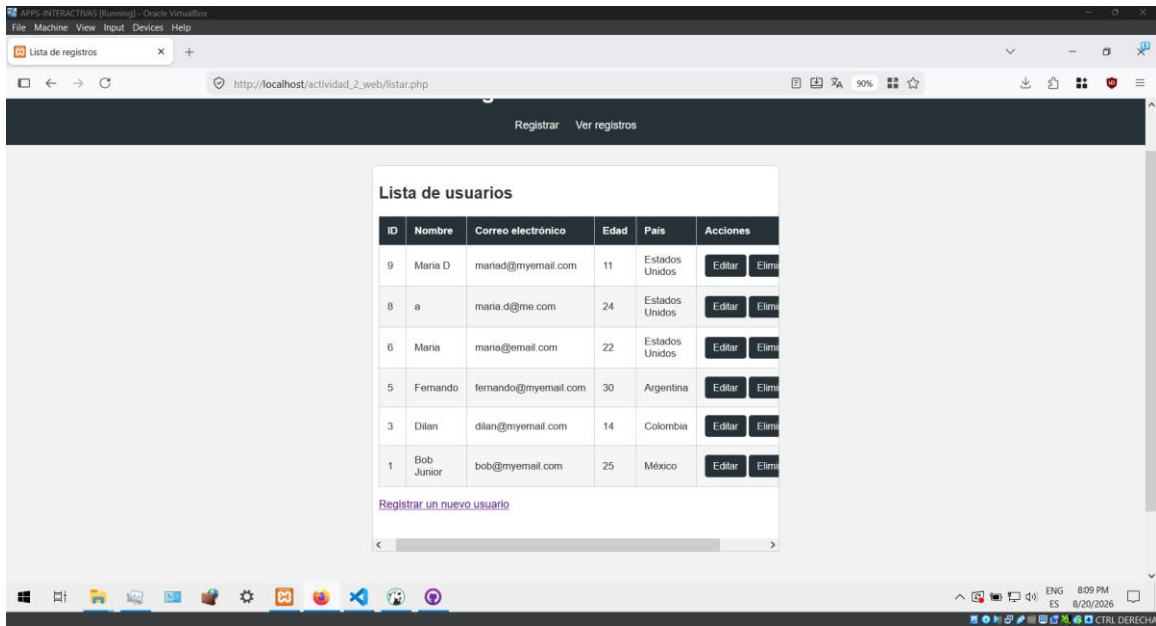


c. Verificación de campos válidos

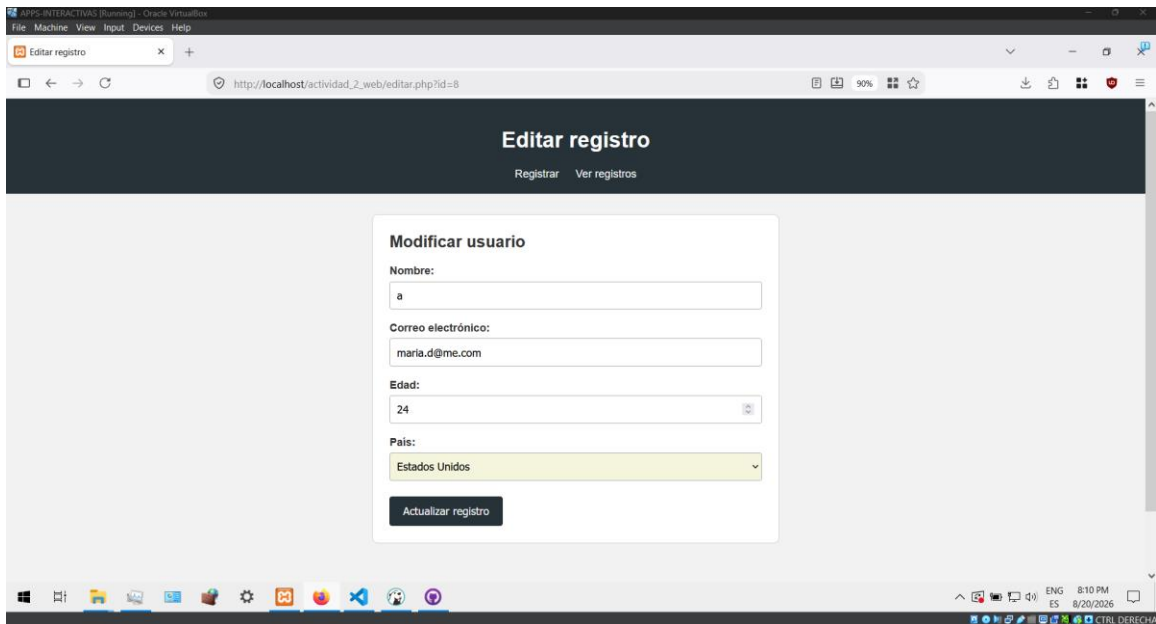
6. Registro de usuarios exitoso



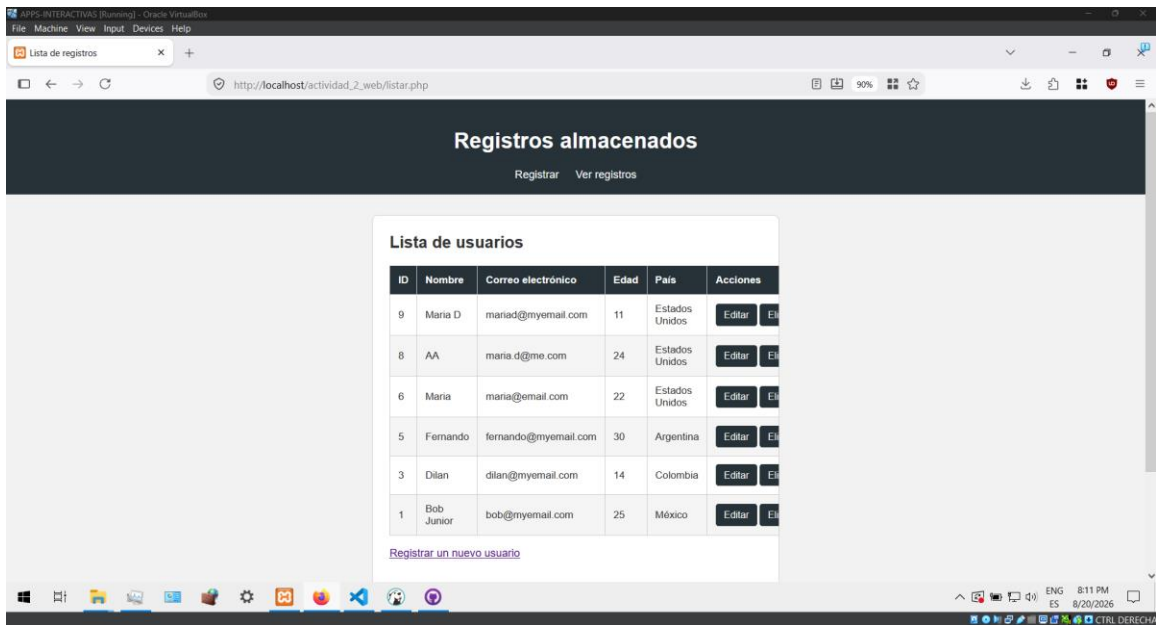
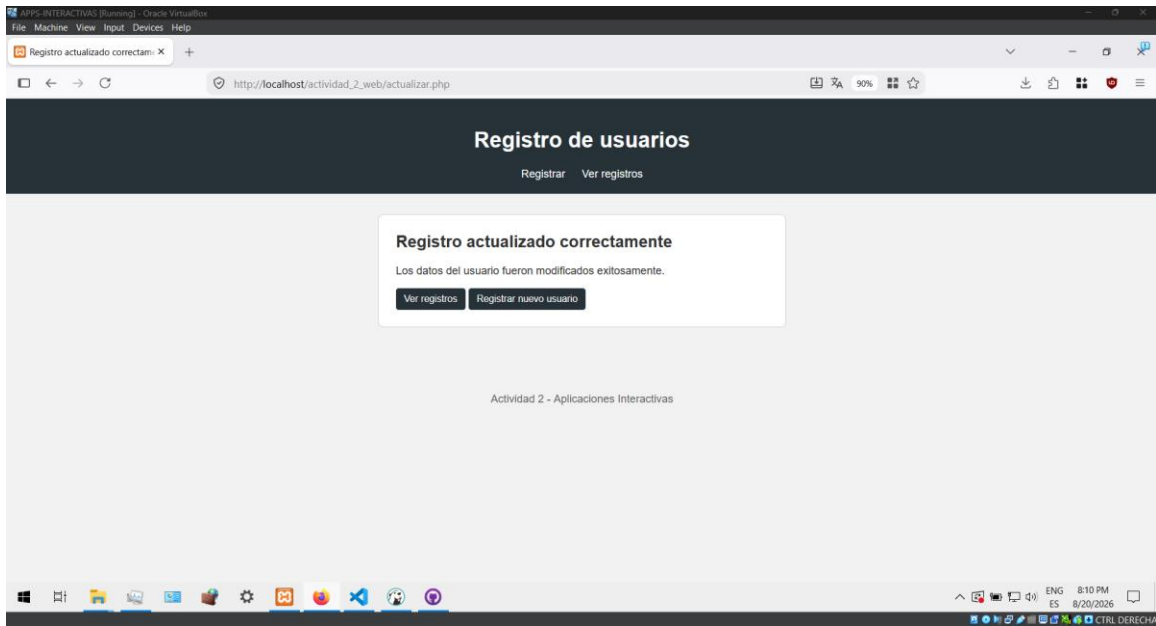
7. Visualización de registros



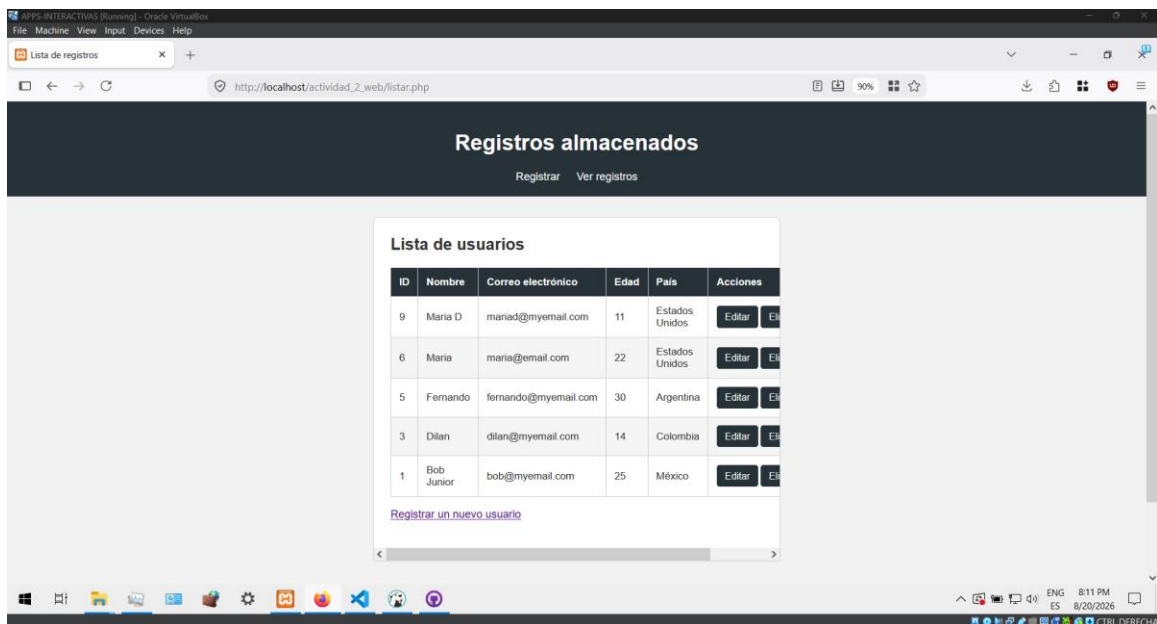
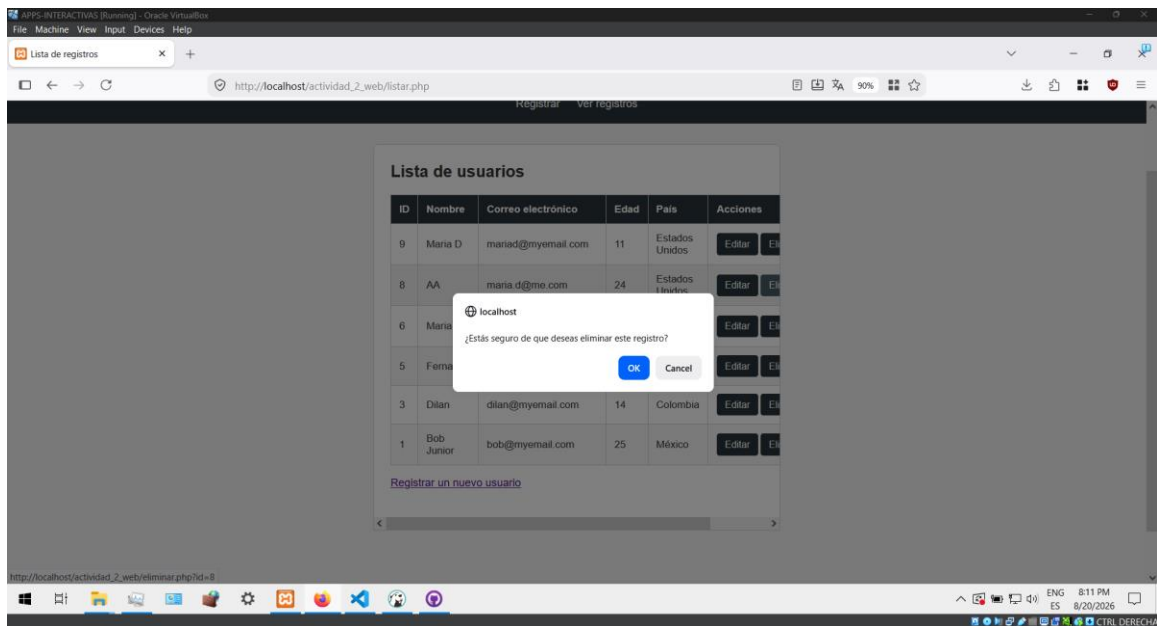
## 8. Editar un registro



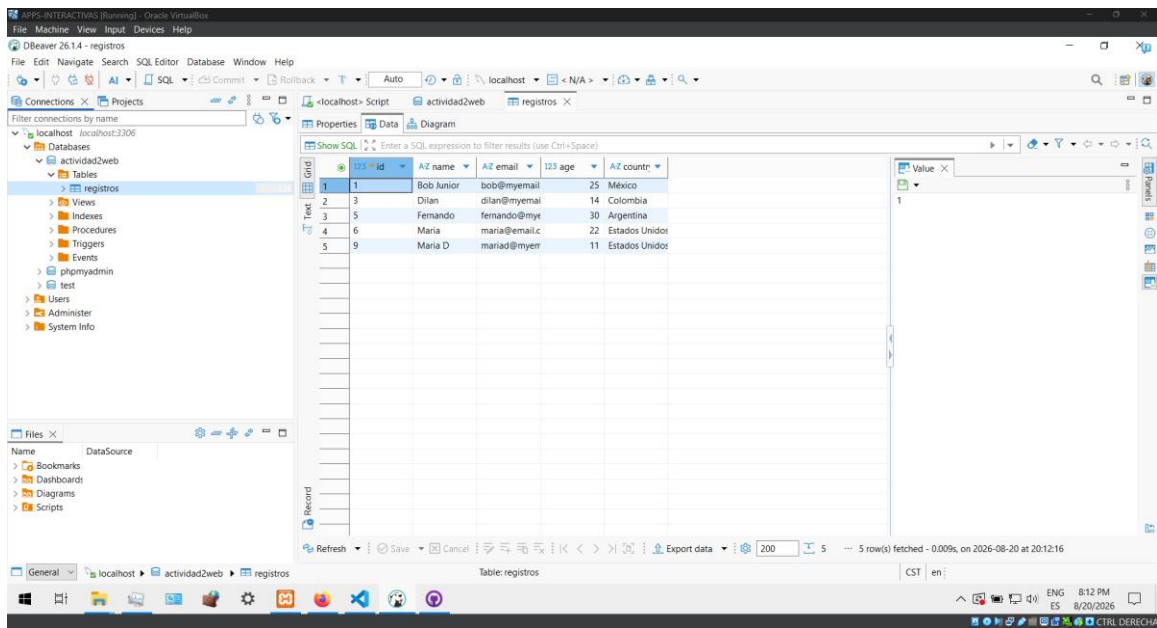
Por ejemplo, de nombre "a", "AA".



## 9. Eliminar un registro



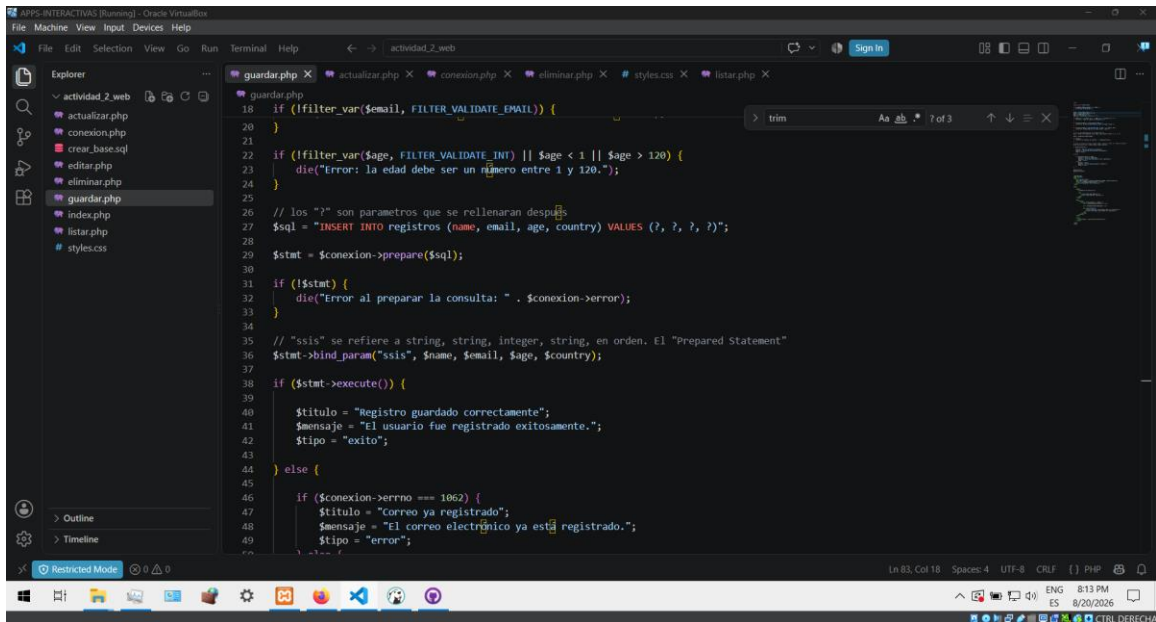
La entrada con nombre "AA" ya no existe.



Los registros se pueden rectificar en DBeaver.

## Ejemplo de uso de sentencias preparadas

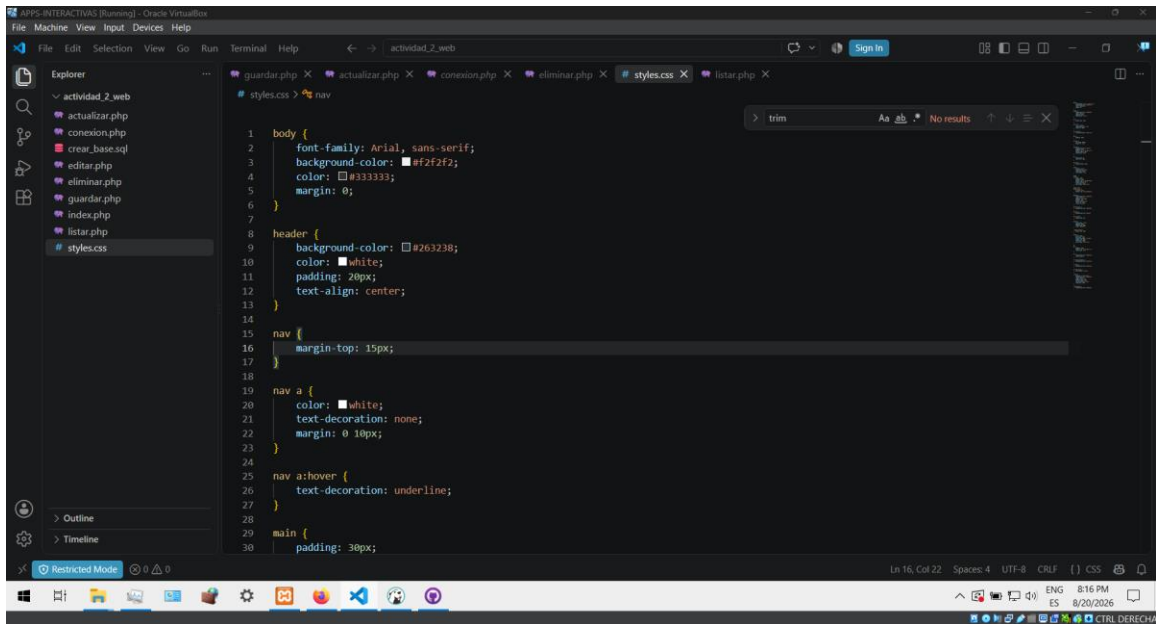
guardar.php utiliza sentencias preparadas como **`$stmt->bind_param("ssis", $name, $email, $age, $country);`** donde "ssis" es string, string, integer, string en orden, y se refiere a las variables, *name*, *email*, *age*, *country* respectivamente que se obtienen del formulario de registro de usuarios. De esta forma, se guardan datos a la BD.



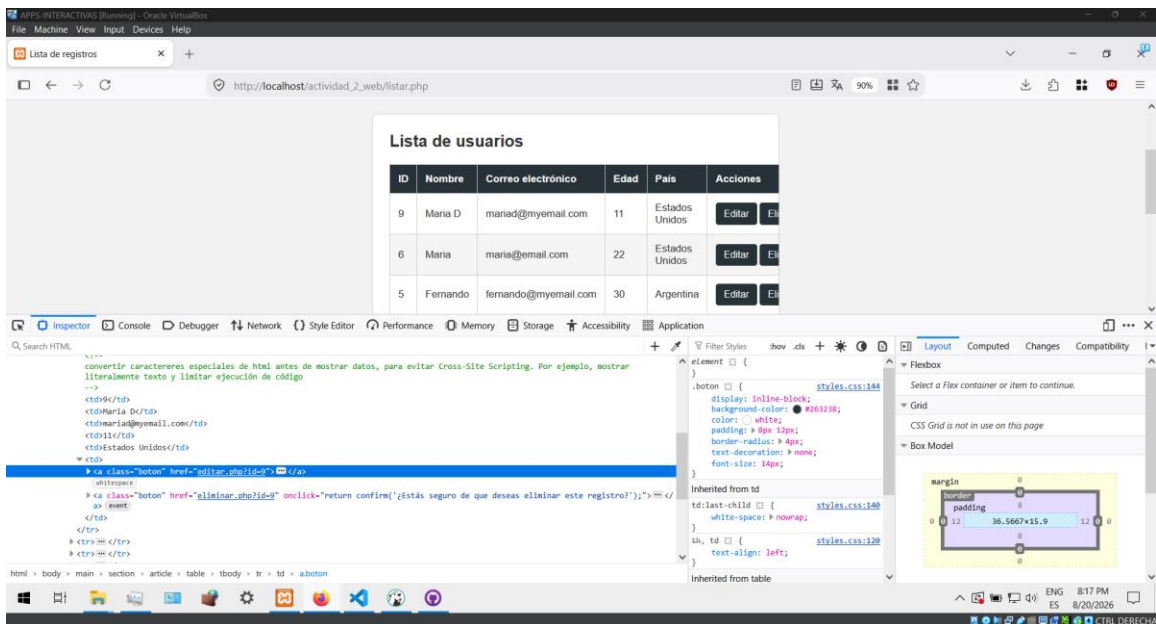
```
guardar.php
18 if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
19 }
20
21
22 if (filter_var($age, FILTER_VALIDATE_INT) || $age < 1 || $age > 120) {
23     die("Error: la edad debe ser un número entre 1 y 120.");
24 }
25
26 // los "?" son parametros que se rellenaran despues
27 $sql = "INSERT INTO registros (name, email, age, country) VALUES (?, ?, ?, ?)";
28
29 $stmt = $conexion->prepare($sql);
30
31 if (!$stmt) {
32     die("Error al preparar la consulta: " . $conexion->error);
33 }
34
35 // "ssis" se refiere a string, string, integer, string, en orden. El "Prepared Statement"
36 $stmt->bind_param("ssis", $name, $email, $age, $country);
37
38 if ($stmt->execute()) {
39
40     $titulo = "Registro guardado correctamente";
41     $mensaje = "El usuario fue registrado exitosamente.";
42     $tipo = "exito";
43 } else {
44
45     if ($conexion->errno === 1062) {
46         $titulo = "Correo ya registrado";
47         $mensaje = "El correo electrónico ya está registrado.";
48         $tipo = "error";
49     }
50 }
```

## Estilo visual

El estilo visual del proyecto se centraliza en `styles.css`, define el formato de tablas, botones, headers, etc.



Por ejemplo, los botones de las acciones de los formularios (en lista de usuarios) utilizan una clase personalizada llamada “boton”.



# Reflexión Final

## **a. ¿Cómo se integran HTML5, CSS3, formularios, PHP y bases de datos para construir una aplicación web dinámica?**

Como visto en la actividad anterior sobre la relación cliente-servidor, HTML5 se encarga de estructurar y mostrar el contenido de la página mediante elementos como header, main, section, form, etc. CSS3 permite darle formato y organizar visualmente estos elementos. Los formularios permiten al usuario introducir información, mientras que PHP recibe y procesa esos datos en el servidor. Finalmente, PHP se comunica con la base de datos para almacenar, consultar, actualizar o eliminar información. De esta manera, todos estos elementos trabajan en conjunto para crear una aplicación web que puede recibir y administrar información proporcionada por el usuario.

## **b. ¿Cómo contribuyen la estructura semántica de HTML5 y el modelo de cajas (Box Model) a la claridad y organización de una interfaz de usuario?**

La estructura semántica de HTML5 permite identificar la función de cada parte de la página mediante etiquetas como header, nav, main, section y footer, lo que facilita la organización del contenido y mejora la accesibilidad. Por ejemplo, los lectores de pantalla pueden anunciar correctamente una página web por su organización. Por otra parte, el Box Model permite controlar el espacio que ocupa cada elemento mediante propiedades como margin, padding y border. Esto ayuda a separar los elementos visualmente y a crear una interfaz más ordenada y fácil de leer.

## **c. ¿Cómo permiten las operaciones CRUD administrar información persistente?**

Las operaciones CRUD permiten administrar la información almacenada de forma persistente en una base de datos. En esta actividad, INSERT permitió crear registros, SELECT consultarlos, UPDATE modificarlos y DELETE

eliminarlos. De esta manera, la aplicación puede mantener y administrar los datos incluso después de cerrar la página.

**d. ¿Qué medidas de validación, sanitización o seguridad aplicaste para proteger los datos?**

Se utilizaron varias medidas para validar y proteger los datos. En el formulario se utilizaron atributos como `required`, `type="email"`, `type="number"` y límites para la edad. En PHP también se validaron los campos recibidos mediante POST, se utilizó `filter_var()` para comprobar el correo y la edad, y `trim()` para eliminar espacios innecesarios. Para evitar ataques de SQL Injection se utilizaron sentencias preparadas (Prepared Statements) con MySQLi. Además, al mostrar información de la base de datos se utilizó `htmlspecialchars()` para evitar que contenido HTML o código introducido por un usuario se interprete directamente en la página.

**e. ¿Qué mejoras implementarías en una futura versión del sistema?**

En una futura versión implementaría un sistema de usuarios con inicio de sesión y diferentes niveles de permisos. También mejoraría el diseño para adaptarlo completamente a dispositivos móviles (diseño responsive) y añadiría mensajes de error más detallados. Además, podría incorporar búsquedas y filtros para facilitar la consulta de registros, así como paginación cuando la cantidad de datos aumente. Finalmente, sería conveniente agregar medidas de seguridad adicionales, como protección CSRF y una configuración más segura de las credenciales de la base de datos.